

Inteligência Computacional

Aula 15: Rede Neural Feed Forward – parte 1

Prof. Eduardo Corrêa Gonçalves

14/10/2026

Sumário

Introdução

O que é rede neural ?

O que é rede neural feed forward ?

Unidades Computacionais

Unidade

Função de ativação

Perceptron e o problema do XOR

Representações escondidas

Rede Neural Feed Forward

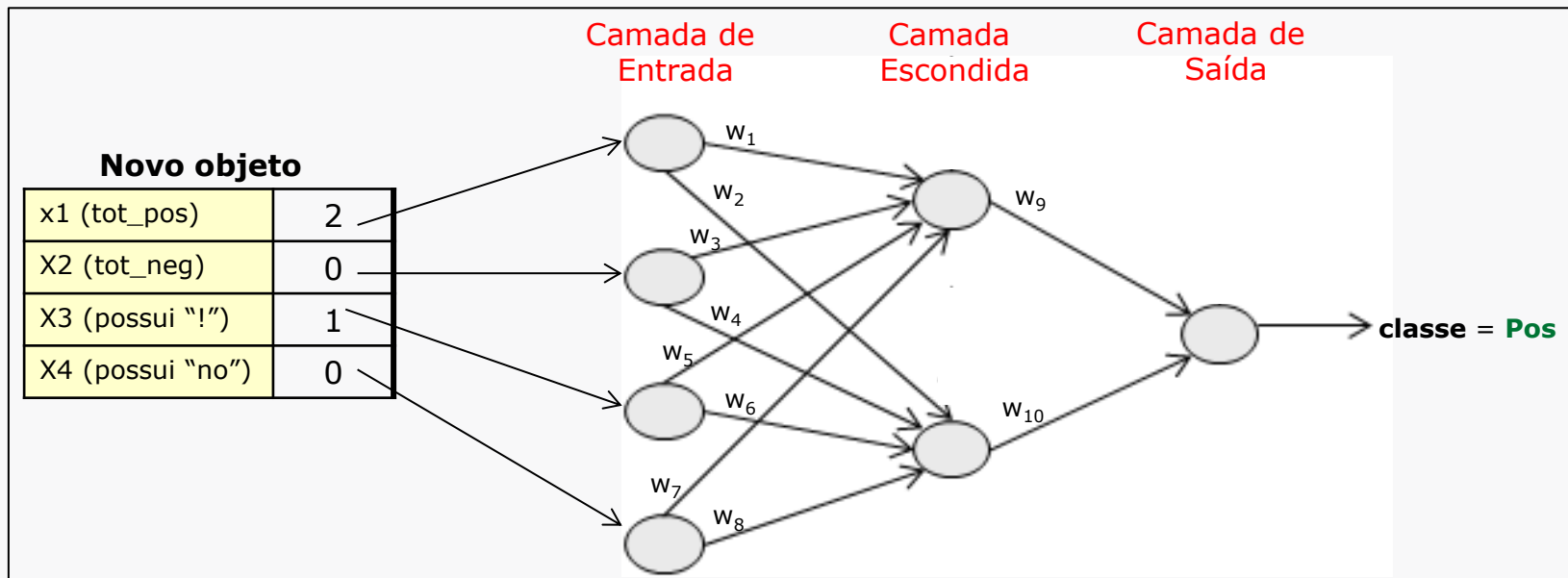
Topologia

Exemplo passo a passo: como classificar um novo objeto

Conhecendo as matrizes de peso

Introdução (1/4)

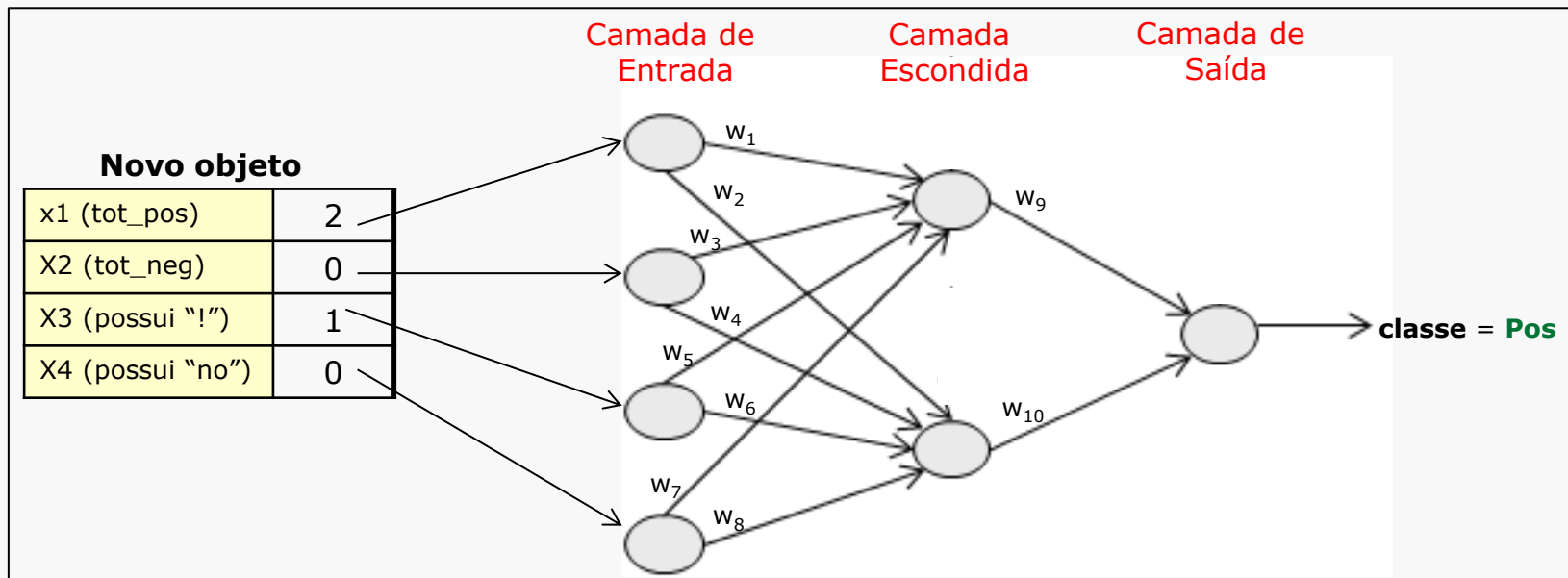
- O que é **Rede Neural** (RN) ?
 - Rede formada por pequenas **unidades conectadas**, estruturadas em **camadas**.
 - Cada conexão possui um **peso** associado (*os pesos são w_1 a w_{10} na figura abaixo*).
 - Pode ser usada para inúmeras tarefas, incluindo classificação.



RN Treinada
(classificador de sentimentos)

Introdução (2/4)

- O que é **Rede Neural FeedForward (RNFF)** ?
 - É o **tipo** de RN mais conhecida e utilizada e mais básica. Mas há outros tipos.
 - O nome vem do fato de que a computação é executada iterativamente de uma camada para a seguinte.



RN Treinada
(classificador de sentimentos)

Introdução (3/4)

- RNFF

- Ao longo das últimas 4 décadas, **RNFFs** têm sido utilizada para a implementação de classificadores.
- Junto com a regressão logística, formam uma excelente porta de entrada para quem deseja começar a estudar sobre aprendizado profundo (*deep learning, área que lida com RNs bem mais complexas*)
- Em comparação com outras técnicas de classificação, as RNs geralmente:
 - Possuem um tempo de treinamento bem maior.
 - Baseiam-se em princípios matemáticos mais sofisticados.
 - Costumam ser altamente eficazes para classificar dados não estruturados (textos, imagens, áudios etc.).

Introdução (4/4)

- Aula de Hoje

- (i) examinar as **propriedades da RNFF**;
- (ii) entender como uma **RNFF já treinada** é utilizada para realizar classificação de texto;
- (iii) entender como são as **matrizes** utilizadas pelas RNs para armazenar os pesos de cada conexão.
- Aspectos relacionados ao **treinamento** da RNFF serão abordados apenas na **última aula** do curso.

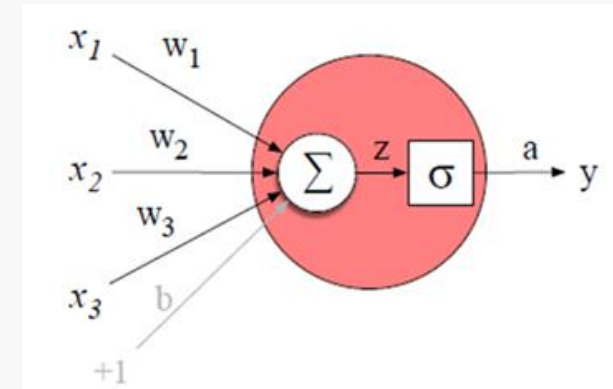
Unidades Computacionais (1/8)

- **Unidade Computacional (ou Unidade Neural)**
 - É o **elemento básico** de uma RN
 - Recebe um conjunto de valores reais como entrada
 - Executa alguma computação sobre eles.
 - Produz uma saída.
 - Em seu núcleo, a unidade computacional realiza:
 - a soma ponderada das entradas (soma cada $w_i x_i$, i.e., cada entrada x_i multiplicada por seu peso w_i) mais o termo de viés b (*bias term*).
 - O resultado z é chamado de entrada (*input*) da unidade.

$$z = \left(\sum_{i=1}^n w_i x_i \right) + b$$

Unidades Computacionais (2/8)

- Exemplo: cálculo da entrada z de uma unidade computacional
- Dada a unidade ao lado, suponha que:
 - $x_1 = 3$; $x_2 = 2$; $x_3 = 0$
 - Podemos representar como o vetor $\mathbf{x} = [3, 2, 0]$
 - $w_1 = 2.5$; $w_2 = -5.0$; $w_3 = 2.0$
 - Podemos representar como o vetor $\mathbf{w} = [2.5, -5.0, 2.0]$
 - $b = 0.1$
- Como calcular o valor de z (entrada da rede) ?

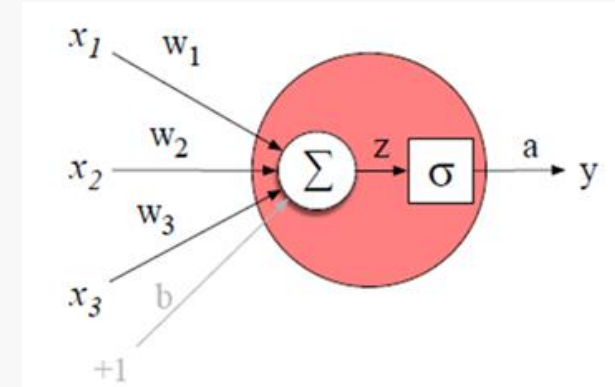


$$z = \left(\sum_{i=1}^n w_i x_i \right) + b$$

Unidades Computacionais (3/8)

• Exemplo: cálculo da entrada z de uma unidade computacional

- $x_1 = 3$; $x_2 = 2$; $x_3 = 0$ $\mathbf{x} = [3, 2, 0]$
- $w_1 = 2.5$; $w_2 = -5.0$; $w_3 = 2.0$ $\mathbf{w} = [2.5, -5.0, 2.0]$
- $b = 0.1$
- $z = 2.5 \times 3 + -5.0 \times 2 + 2.0 \times 0 + 0.1 = \underline{\underline{-2.4}}$



$$z = \left(\sum_{i=1}^n w_i x_i \right) + b$$

Unidades Computacionais (4/8)

- Função de Ativação

- Porém... na RN, uma dada unidade **não usa z** , função linear de x , como saída.
- Em vez disso, aplica uma **função não linear σ** a z .
- A saída é chamada de **valor de ativação** da unidade (representado por a na figura abaixo)

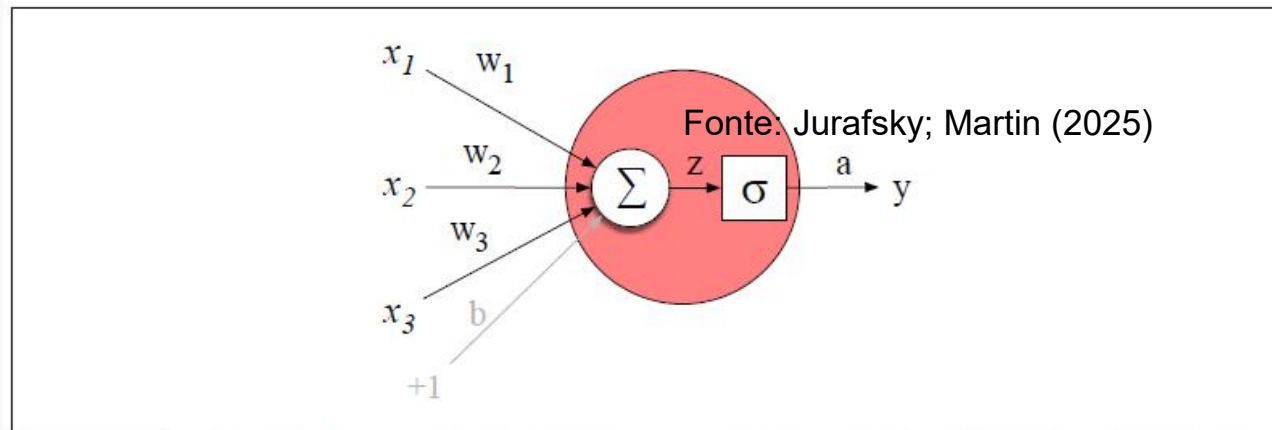


Figure 7.2 A neural unit, taking 3 inputs x_1 , x_2 , and x_3 (and a bias b that we represent as a weight for an input clamped at $+1$) and producing an output y . We include some convenient intermediate variables: the output of the summation, z , and the output of the sigmoid, a . In this case the output of the unit y is the same as a , but in deeper networks we'll reserve y to mean the final output of the entire network, leaving a as the activation of an individual node.

Unidades Computacionais (5/8)

- **Função Linear**

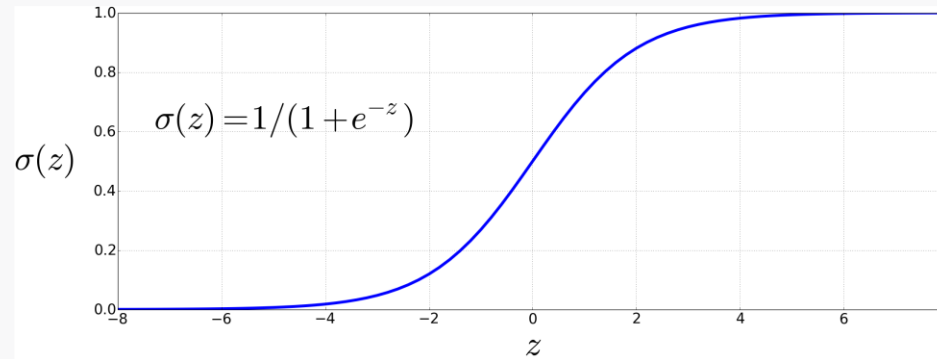
- Uma função linear é uma função que tem um hiperplano como solução.
 - Pense em um hiperplano como uma superfície plana, sem curvatura...
 - (*alguma “coisa” linear é algo que está relacionado a uma reta*).
 - **Ex.:** $5x_1 + 3x_2 - 4x_3 + 1 = 0$

- **Função Não Linear**

- Qualquer função que não seja linear...
 - Qualquer função que não represente uma relação linear entre as variáveis de entrada e a saída
 - **Ex.:** funções trigonométricas, exponenciais, logarítmicas etc.

Unidades Computacionais (6/8)

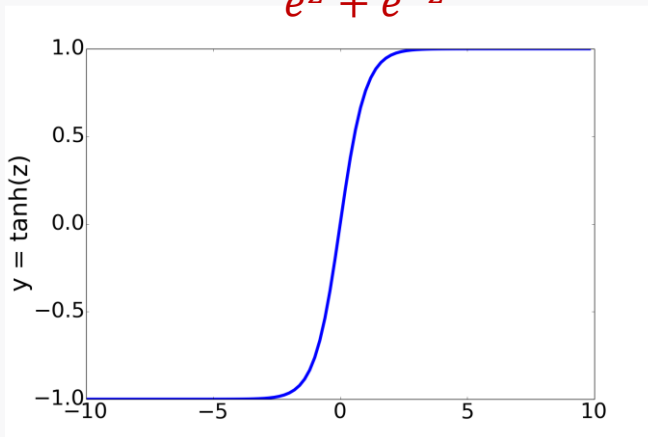
- Funções de ativação muito usadas nas RNs (*todas são não lineares*)



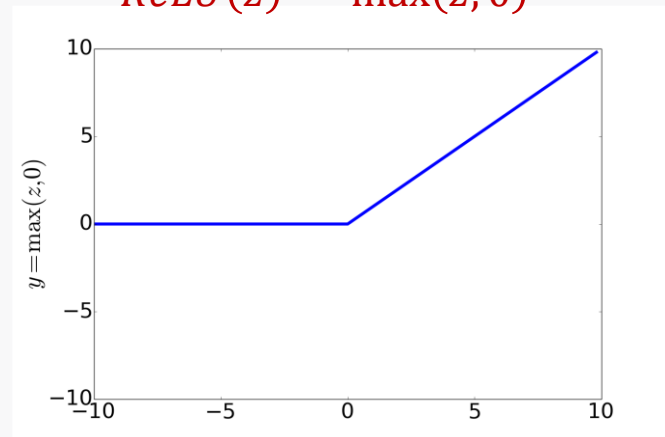
$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

(sigmoide)

$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$

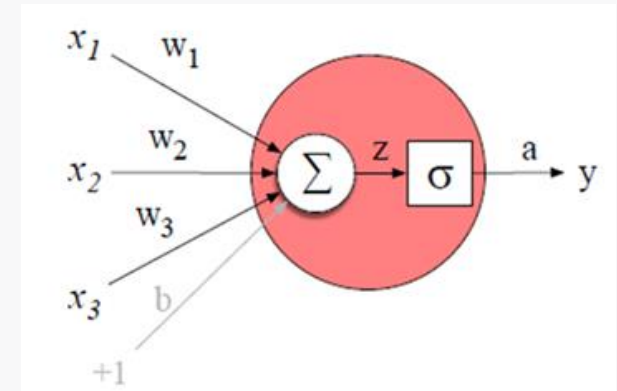


$$\text{ReLU}(z) = \max(z, 0)$$



Unidades Computacionais (7/8)

- Exemplo: cálculo da entrada e da saída de uma unidade computacional
- Vamos retomar o exemplo anterior. Dada a unidade ao lado, suponha que:
 - $x_1 = 3$; $x_2 = 2$; $x_3 = 0$
 - Podemos representar como o vetor $\mathbf{x} = [3, 2, 0]$
 - $w_1 = 2.5$; $w_2 = -5.0$; $w_3 = 2.0$
 - Podemos representar como o vetor $\mathbf{w} = [2.5, -5.0, 2.0]$
 - $b = 0.1$
- Considere ainda que vamos usar a **sigmoide** como função de ativação.



$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Unidades Computacionais (8/8)

- Exemplo: cálculo da entrada e da saída de uma unidade computacional

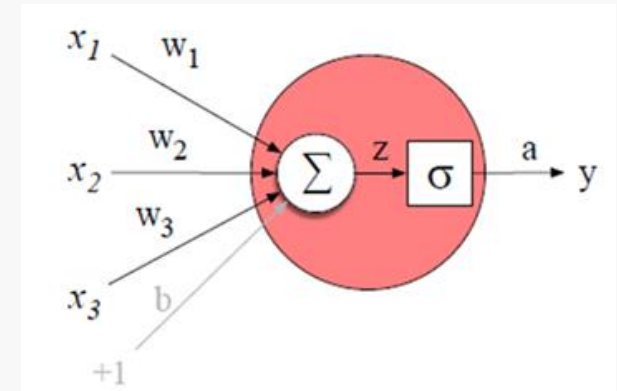
- $\mathbf{x} = [3, 2, 0]$; $\mathbf{w} = [2.5, -5.0, 2.0]$; $b = 0.1$

- Cálculo da entrada z :

- $z = 3 \times 2.5 + 2 \times -5.0 + 0 \times 2.0 + 1 \times 0.1 = -2.4$

- Cálculo da saída a :

- $a = \sigma(z) = 1 / (1 + e^{-z}) = 1 / (1 + e^{-(-2.4)}) = \underline{\underline{0.0832}}$



$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

* O número e na fórmula da sigmoide é o número de Euler, que corresponde a constante matemática $\approx 2,7182818$

Combinando Unidades Computacionais (1/7)

- **O poder de combinar unidades**
 - Desde o início da história das RNs, foi percebido que seu **poder** é obtido pela **combinação das unidades** em **redes maiores**.
 - Assim como ocorre com os neurônios do cérebro humano.
 - Uma demonstração foi dada em 1969, por Minsky e Papert:
 - “*Uma única unidade neural não é capaz de computar certas funções elementares, por exemplo, a função XOR*”

AND			OR			XOR		
x1	x2	y	x1	x2	y	x1	x2	y
0	0	0	0	0	0	0	0	0
0	1	0	0	1	1	0	1	1
1	0	0	1	0	1	1	0	1
1	1	1	1	1	1	1	1	0

Combinando Unidades Computacionais (2/7)

- **Perceptron**

- Esse problema foi apresentado originalmente para o **Perceptron**.
- O Perceptron é uma unidade neural extremamente simples.
 - Não usa função de ativação.
 - Possui saída binária.

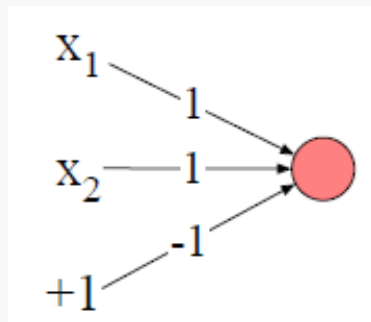
- $y = \begin{cases} 0, & \text{se } \mathbf{wx} + b \leq 0 \\ 1, & \text{se } \mathbf{wx} + b > 0 \end{cases}$

Combinando Unidades Computacionais (3/7)

- Resolvendo a função AND com um Perceptron
 - É muito fácil construir um perceptron para a função AND.

AND		
x1	x2	y
0	0	0
0	1	0
1	0	0
1	1	1

Função AND



Perceptron AND

x1	x2	Entrada Perceptron (z)	Saída Perceptron (y)
0	0	$1 \times 0 + 1 \times 0 - 1 = -1$	0
0	1	$1 \times 0 + 1 \times 1 - 1 = 0$	0
1	0	$1 \times 1 + 1 \times 0 - 1 = 0$	0
1	1	$1 \times 1 + 1 \times 1 - 1 = 1$	1

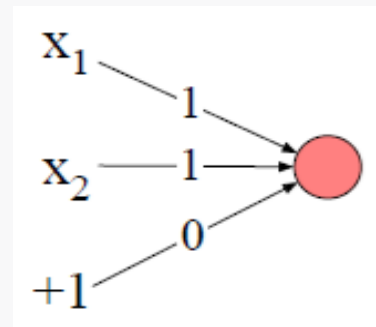
Combinando Unidades Computacionais (4/7)

- Resolvendo a função OR com um Perceptron

- Também é muito fácil construir um perceptron para a função OR.

OR		
x1	x2	y
0	0	0
0	1	1
1	0	1
1	1	1

Função OR



Perceptron OR

x1	x2	Entrada Perceptron (z)	Saída Perceptron (y)
0	0	$1 \times 0 + 1 \times 0 + 0 = 0$	0
0	1	$1 \times 0 + 1 \times 1 + 0 = 1$	1
1	0	$1 \times 1 + 1 \times 0 + 0 = 1$	1
1	1	$1 \times 1 + 1 \times 1 + 0 = 1$	1

- Lembre que a saída Perceptron é computada por:

- $y = 0$, se $\mathbf{wx} + b \leq 0$
- $y = 1$, se $\mathbf{wx} + b > 0$

Combinando Unidades Computacionais (5/7)

• O problema do XOR

- Porém, **não é possível** construir um Perceptron para o **XOR** !!!
- **Motivo:** o Perceptron é um classificador linear
- Não há como desenhar uma linha que separe os casos positivos do XOR (01 e 10) dos casos negativos (00 e 11).

XOR		
x1	x2	y
0	0	0
0	1	1
1	0	1
1	1	0

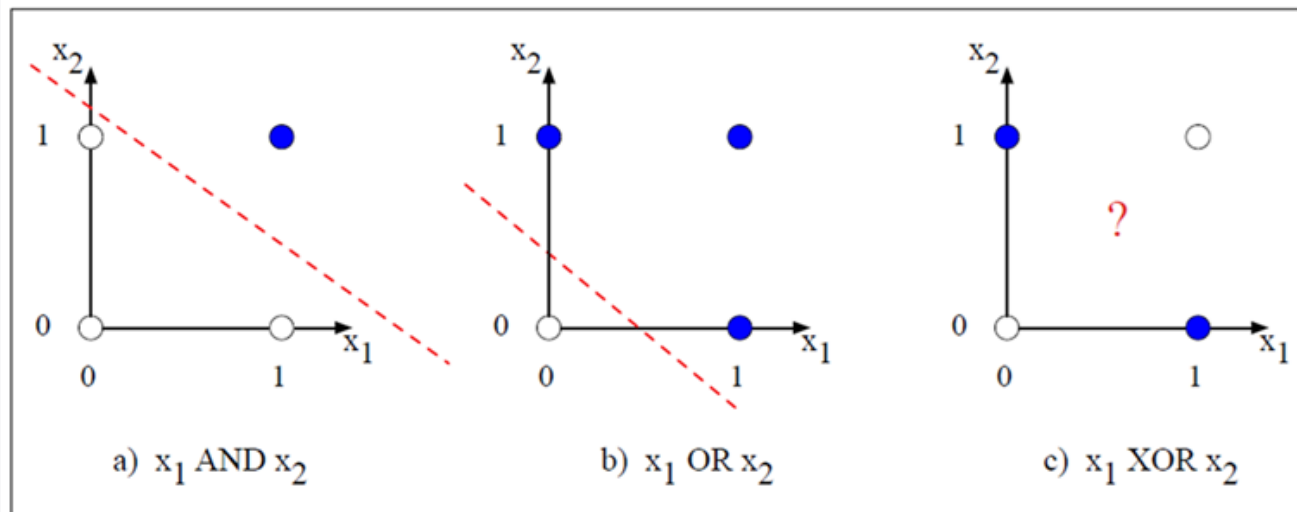
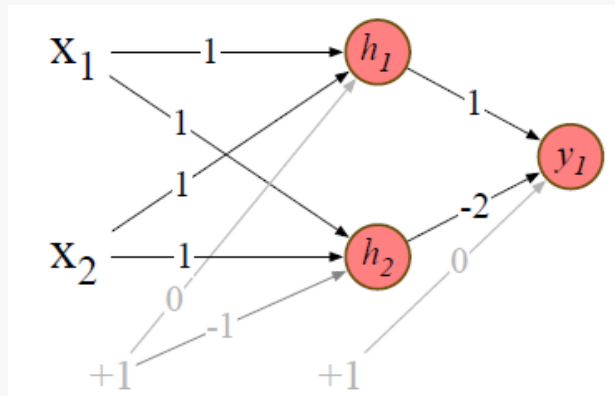


Figure 7.5 The functions AND, OR, and XOR, represented with input x_1 on the x-axis and input x_2 on the y-axis. Filled circles represent perceptron outputs of 1, and white circles perceptron outputs of 0. There is no way to draw a line that correctly separates the two categories for XOR.

Fonte: Jurafsky;
Martin (2025)

Combinando Unidades Computacionais (6/7)

- O problema do XOR – A solução é a **rede neural** !!!
 - XOR não pode ser calculado por um único Perceptron. Mas pode por uma RN em camadas formada por unidades computacionais.
 - **Exemplo:** usando 3 unidades ReLU h_1 , h_2 e y_1 dispostas em 2 camadas.



x1	x2	entr. h1	entr. h2	saída h1	saída h2	entr. y1	saída y1
0	0	0	-1	0	0	0	0
0	1	1	0	1	0	1	1
1	0	1	0	1	0	1	1
1	1	2	1	2	1	0	0

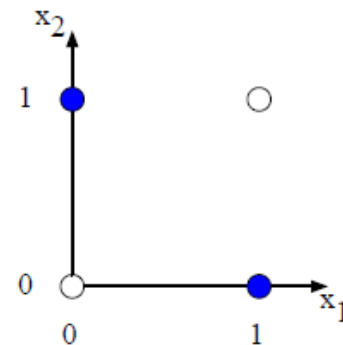
- Saída h_1 , h_2 e y_1 :
 - **$\text{ReLU}(z) = \max(z, 0)$**

Combinando Unidades Computacionais (7/7)

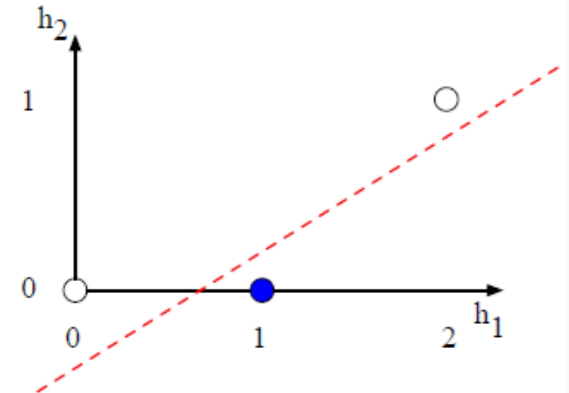
• Representações Escondidas

- No exemplo, a **camada escondida** da RN formou uma **nova representação** da entrada original.
 - As representações escondidas dos pontos $x = [0, 1]$ e $x = [1, 0]$ (os casos positivos do XOR) são combinadas para o ponto $[1, 0]$.
 - Essa operação faz com que seja fácil separar linearmente os casos positivos e negativos do XOR.

saída h1	saída h2	entr. y1
0	0	0
1	0	1
1	0	1
2	1	0



a) The original x space

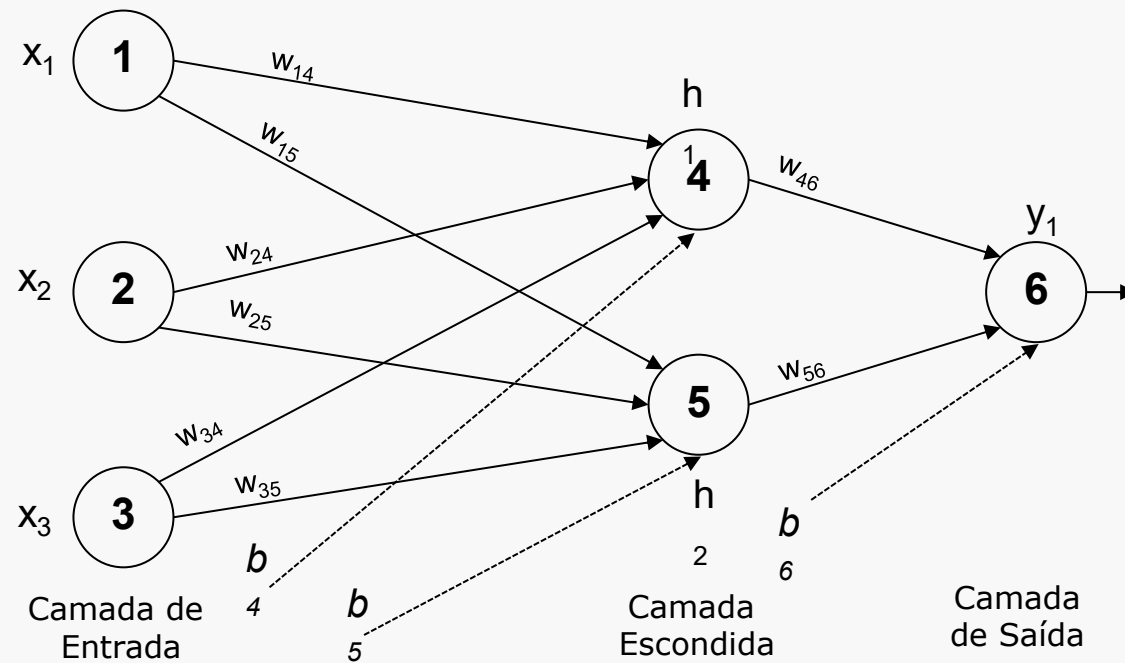


b) The new (linearly separable) h space

Classificação com uma RNFF (1/10)

- Características da RNFF

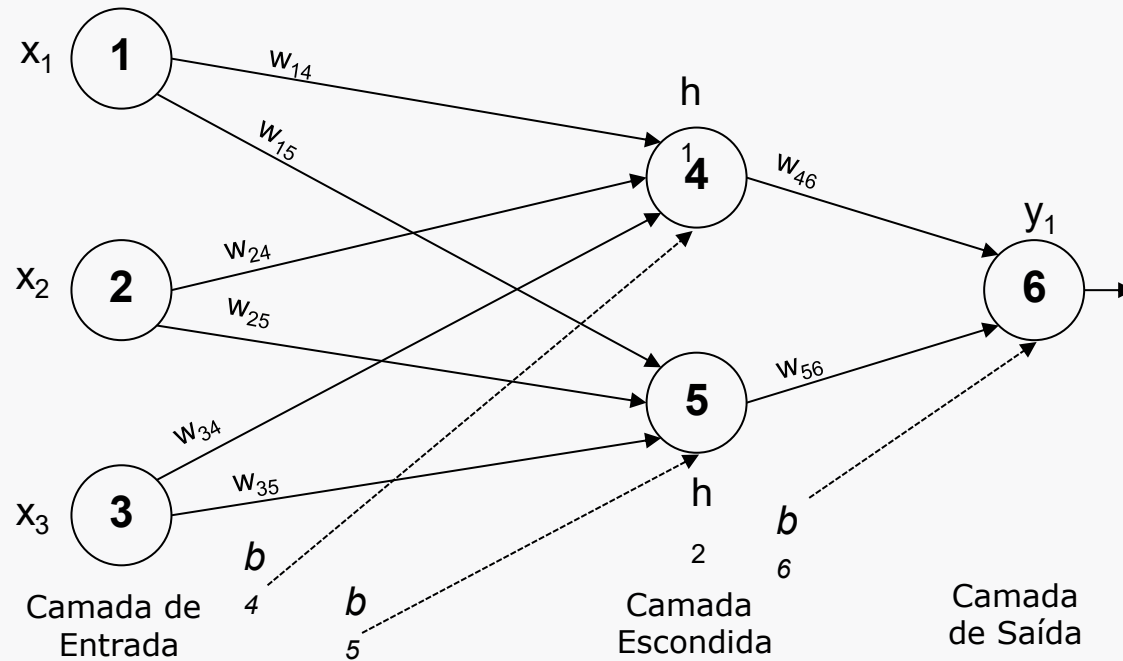
- A RNFF é uma RN multicamadas em que as unidades são conectadas **sem ciclos**.
- O fluxo da informação é apenas “para frente” (*forward*), i.e., as saídas das unidades de uma camada são passadas para unidades da próxima camada (camada mais alta).



Classificação com uma RNFF (2/10)

• Características da RNFF

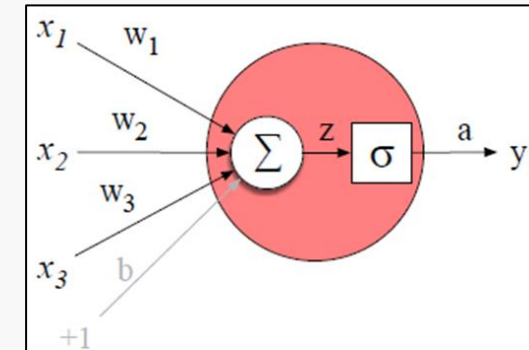
- Na arquitetura padrão, cada camada é **completamente conectada**.
 - **Ou seja:** cada unidade em cada camada recebe como entrada as saídas de todas as unidades da camada anterior.
 - Mas a conexão completa não é algo obrigatório.



Classificação com uma RNFF (3/10)

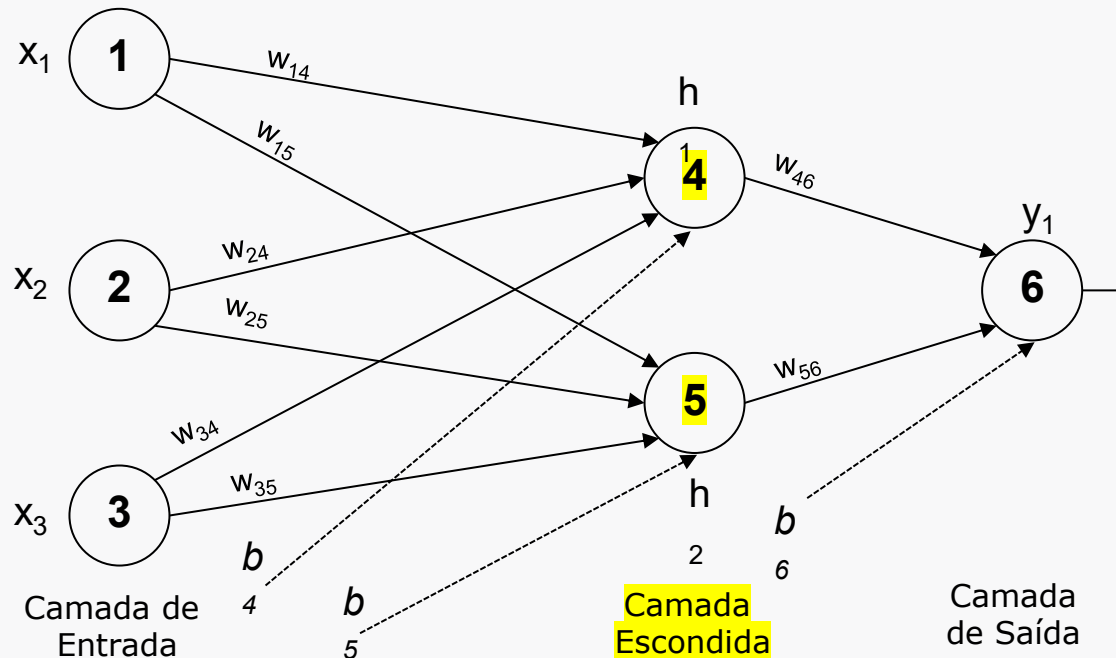
- **FeedForward ou Multi-Layer Perceptron**

- Por razões históricas, a RNFF é também chamada de Perceptron Multi-Camadas (**Multi-Layer Perceptron – MLP**).
- Isso ocorre inclusive em softwares conhecidos, como a sklearn.
- O nome é um pouco equivocado, já que as unidades das RNs atuais não são o “velho” Perceptron
- Como vimos no exemplo do XOR, o Perceptron é puramente linear, enquanto as **unidades neurais atuais** não são assim.



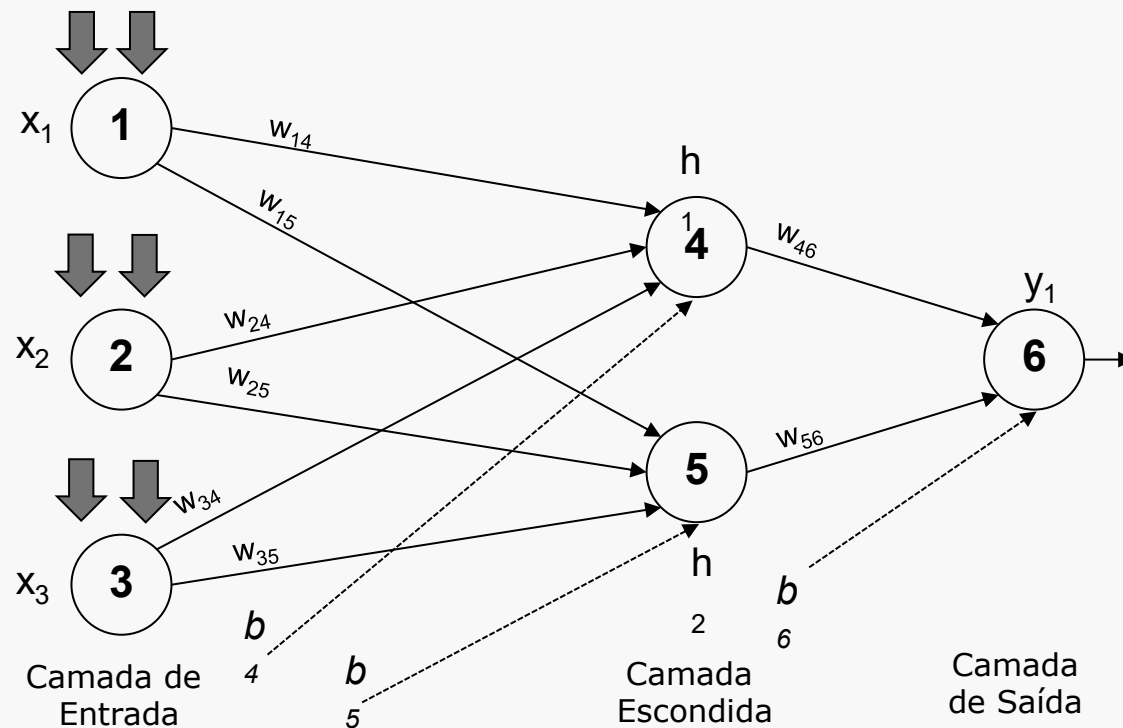
Classificação com uma RNFF (4/10)

- **Camada Escondida (*hidden layer*)** → é a camada mais importante de uma RNFF, composta pelas *hidden units*.
- A RNFF abaixo possui apenas 1 camada escondida, porém **é possível** definir uma RNFF com **mais camadas escondidas**.
 - É chamada de **rede 2-camadas** (1 camada escondida + 1 camada de saída... a camada de entrada não conta).



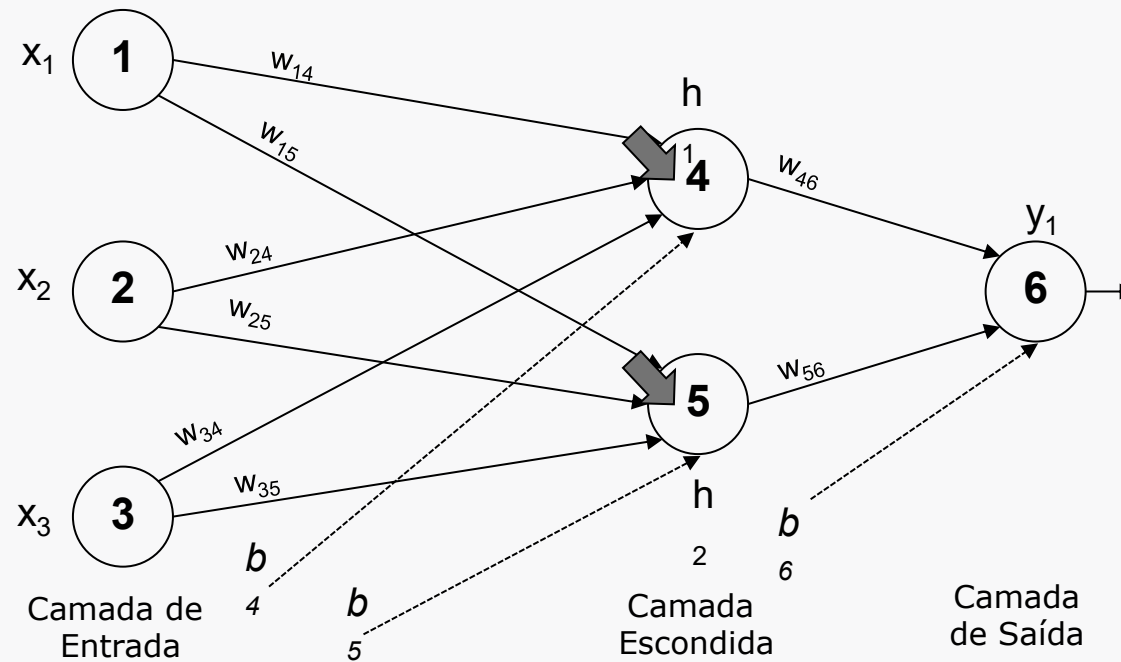
Classificação com uma RNFF (5/10)

- Como um novo objeto é classificado?
 - **PASSO 1 – entrada do novo objeto na rede:** o novo objeto x é passado como um **vetor numérico** através da camada de entrada: $x = [x_1, x_2, x_3]$
 - Ele vai sair da camada de entrada do mesmo jeito que entrou (com os mesmos valores)



Classificação com uma RNFF (6/10)

- Como um novo objeto é classificado?
 - **PASSO 2 – entrada dos nós escondidos:** vetor x é multiplicado pelos pesos de cada aresta w_{ij} , adicionando o vetor de vies correspondente b_j .
 - $I_4 = x_1w_{14} + x_2w_{24} + x_3w_{34} + b_4$ *I_4 significa input 4 (entrada da unidade 4)*
 - $I_5 = x_1w_{15} + x_2w_{25} + x_3w_{35} + b_5$ *I_5 significa input 5 (entrada da unidade 5)*



Classificação com uma RNFF (7/10)

- Como um novo objeto é classificado?

- **PASSO 3 – saída dos nós escondidos:** aplica-se uma função de ativação (ex.: sigmoide) sobre o valor computado.

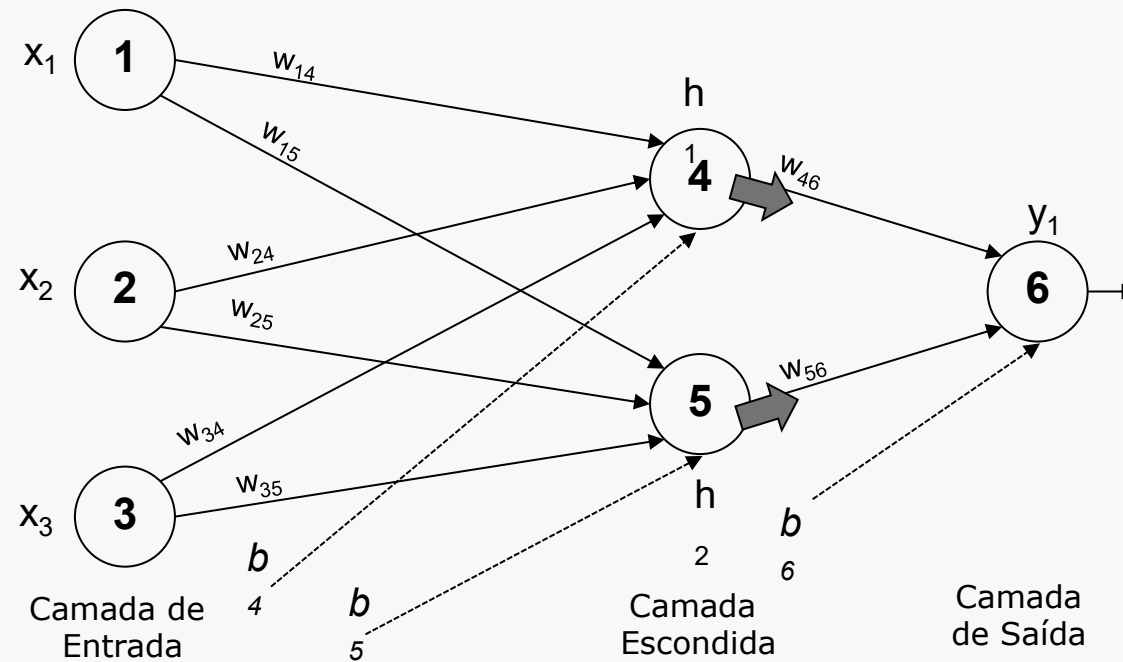
- **Obs.:** É comum usar σ para representar a função de ativação, mesmo que seja outra que não a sigmoide

- $O_4 = \sigma(I_4)$

- $O_5 = \sigma(I_5)$

O_4 significa output 4 (saída da unidade 4)

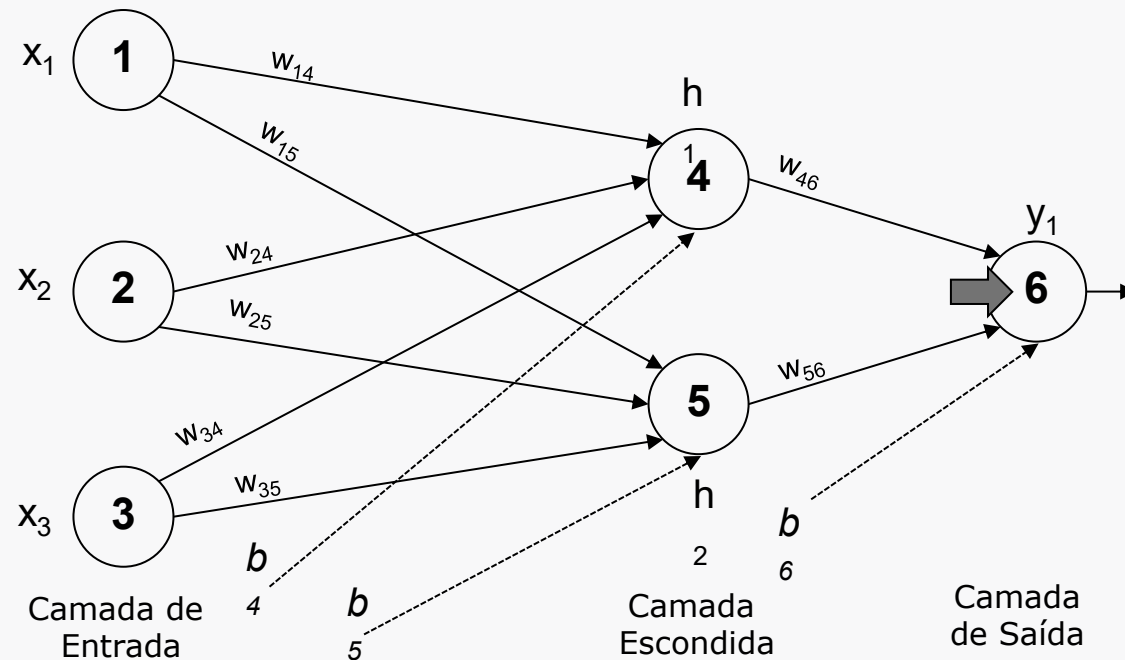
O_5 significa output 5 (saída da unidade 5)



Classificação com uma RNFF (8/10)

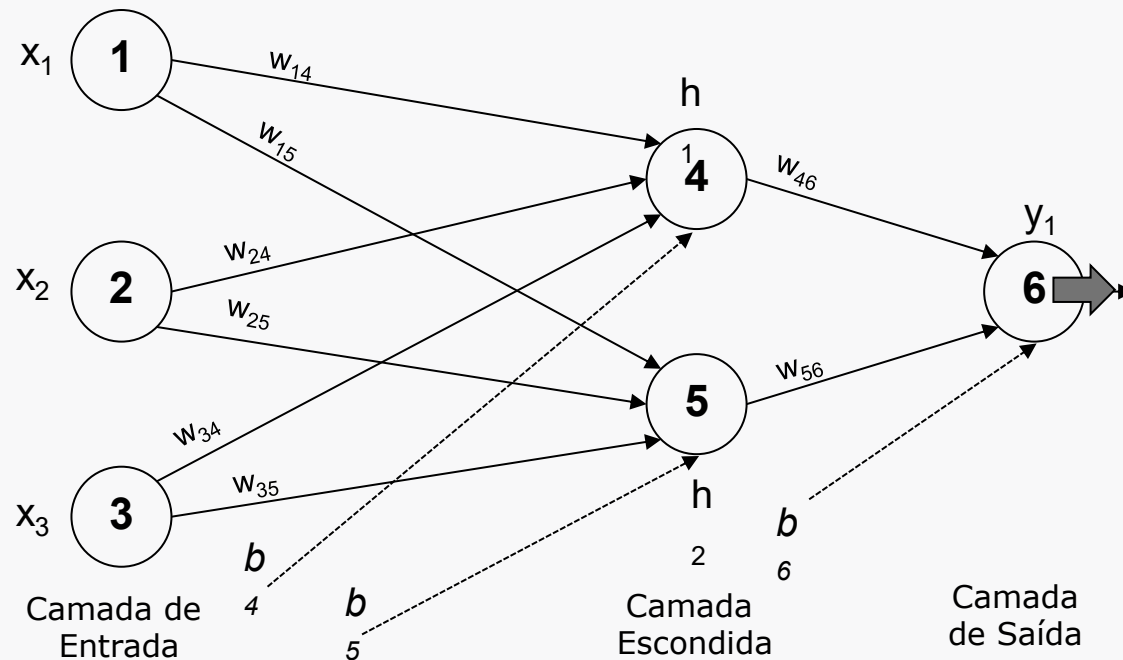
- Como um novo objeto é classificado?
 - **PASSO 4 – entrada do nó de saída:** as saídas O_i de cada nó i da camada escondida são multiplicadas pelos pesos de cada aresta w_{ij} , adicionando o vetor de viés b_j

- $I_6 = O_4 w_{46} + O_5 w_{56} + b_6$



Classificação com uma RNFF (9/10)

- Como um novo objeto é classificado?
 - **PASSO 5** – saída do nó da camada de saída: aplica-se uma função de ativação (ex.: sigmoide) sobre o valor computado.
 - $O_6 = \sigma(I_6)$
 - *A partir desse valor, obtém-se a classificação final (como veremos logo logo)*



Classificação com uma RNFF (10/10)

- **Importante !**

- O exemplo apresentou uma **RNFF** para **classificação binária** que, por isso, possui **1 unidade de saída**.
 - Veremos em uma aula futura o caso da classificação multiclasse, onde existirá mais de uma unidade na camada de saída.
- Essa RNFF possui **1 camada escondida**.
 - Nesse caso, a classificação pode ser vista como a construção de um **vetor h** (hidden), que é a **representação escondida** da entrada.
 - No exemplo apresentado: $h = [O_4, O_5]$
 - Entretanto, existem RNs com muitas camadas escondidas (de centenas a milhares).
 - Elas são chamadas *deep neural networks** (se tiver mais de 1 escondida, já é chamada de *deep*)

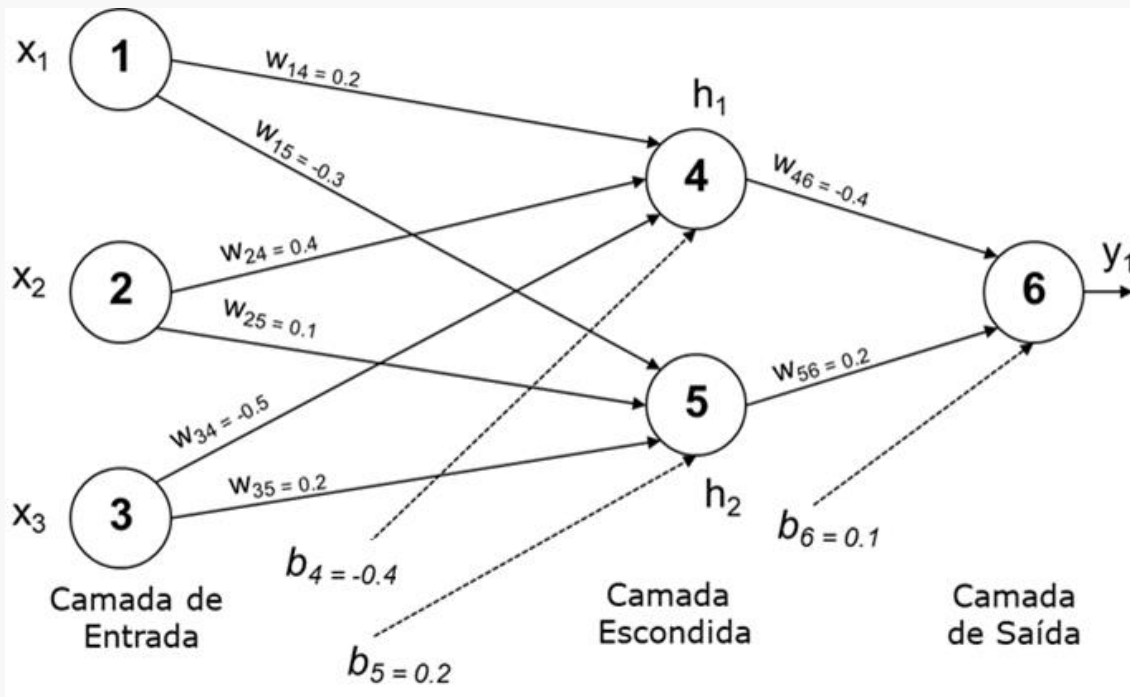
* <https://www.ibm.com/think/topics/deep-learning>

Matrizes da RNFF (1/13)

- **Representação matricial**

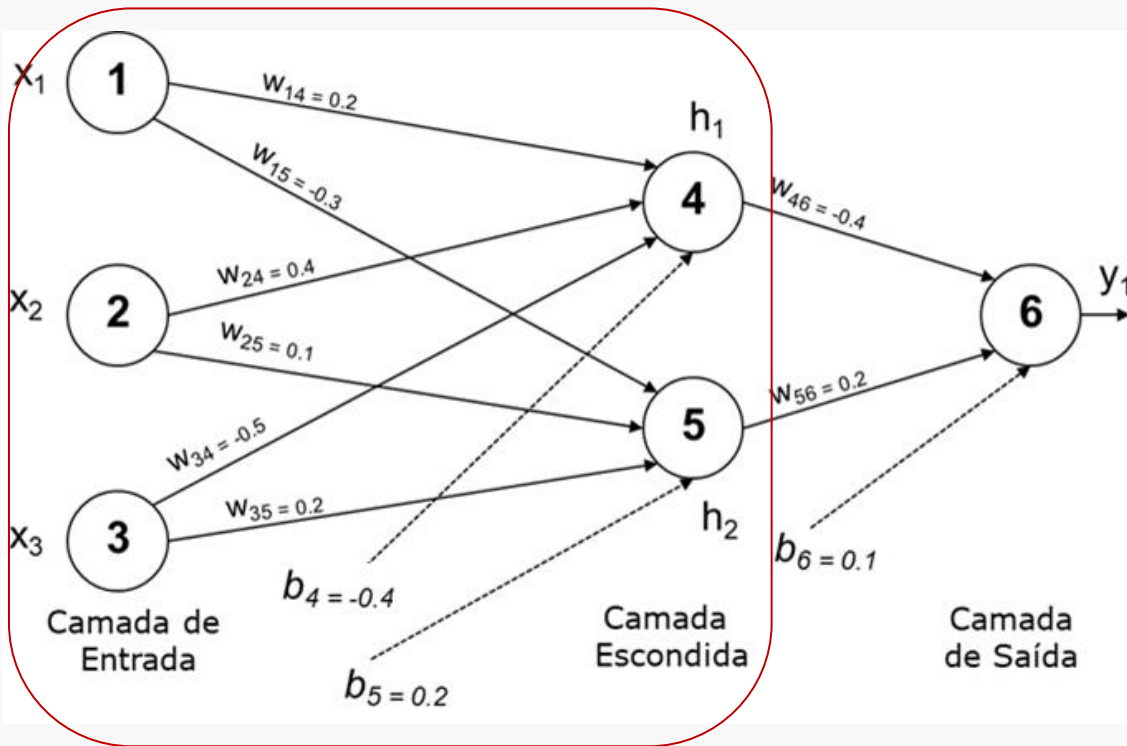
- Na prática, os softwares que implementam as RNs (como a sklearn) representam os parâmetros (pesos e termos de viés) em matrizes e vetores.
- Isso torna o processo de treinamento e classificação **mais eficiente**, pois as operações com matrizes podem ser **paralelizadas** pelas CPUs e GPUs modernas.
- Nos próximos slides vamos introduzir a notação para a representação matricial de RNs utilizando uma rede com a mesma topologia da mostrada no exemplo anterior

Matrizes da RNFF (2/13)



- **Notação:**
- n_0 = número de unidades na camada de entrada → 3
- n_1 = número de unidades na camada escondida → 2
- n_2 = número de unidades na camada de saída → 1

Matrizes da RNFF (3/13)



- $n_o = 3$

- $n_1 = 2$

- $n_2 = 1$

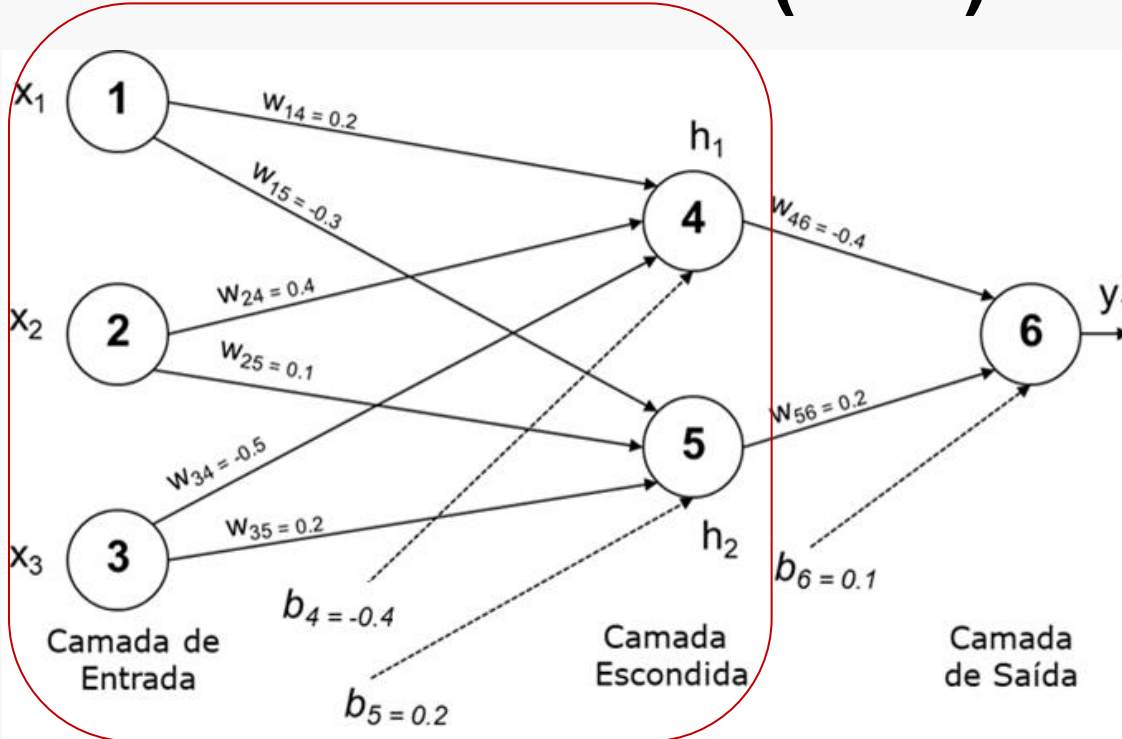
- **Parâmetros da camada escondida**

- **x**: o vetor coluna de entrada $\rightarrow$ dim. $n_o \times 1$

- **W**: matriz de pesos, em que cada linha armazena os pesos das conexões das n_o unidades de entrada para uma unidade de saída $h_j \rightarrow$ dim. $n_1 \times n_o$

- **$b^{[1]}$** : o vetor coluna de pesos da camada escondida $\rightarrow$ dim. $n_1 \times 1$

Matrizes da RNFF (4/13)



- $n_o = 3$
- $n_1 = 2$
- $n_2 = 1$
- $\mathbf{x} = n_o \times 1$
- $\mathbf{W} = n_1 \times n_o$
- $\mathbf{b}^{[1]} = n_1 \times 1$

- **Parâmetros da camada escondida**

$\mathbf{x}$

$\mathbf{W}$
0.2 0.4 -0.5
-0.3 0.1 0.2

→ pesos de h_1

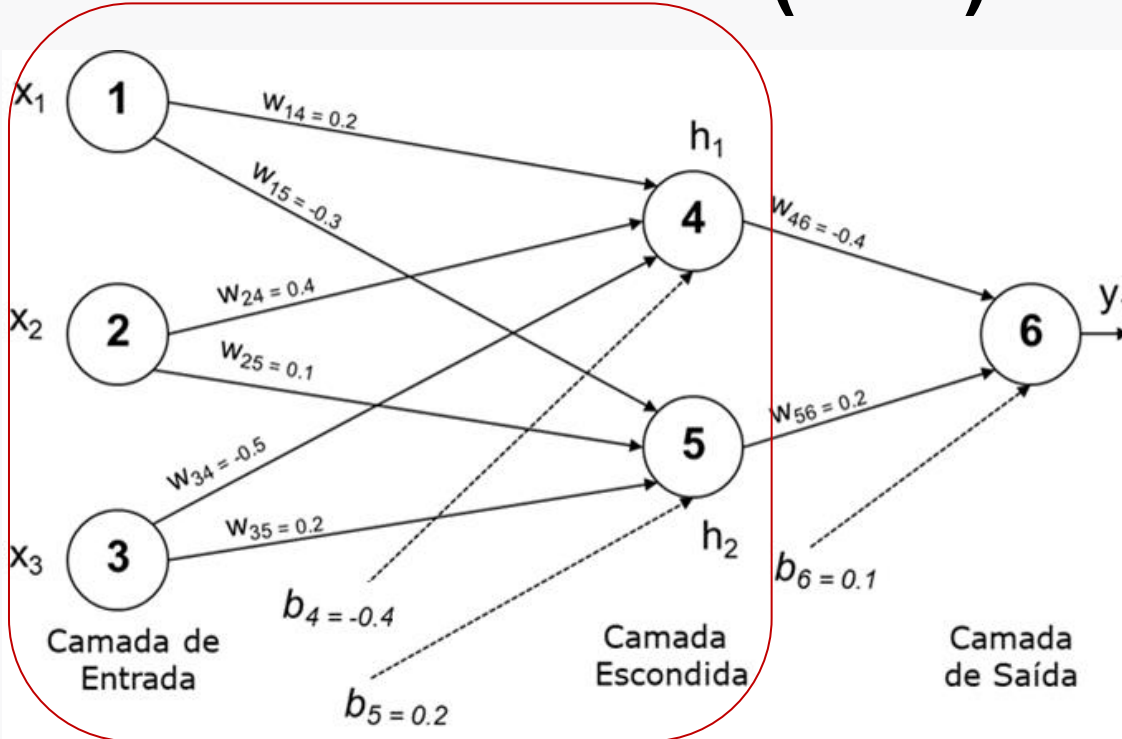
→ pesos de h_2

$\mathbf{b}^{[1]}$
-0.4
0.2

→ bias de h_1

→ bias de h_2

Matrizes da RNFF (5/13)



- $n_o = 3$
- $n_1 = 2$
- $n_2 = 1$
- $\mathbf{x} = n_o \times 1$
- $\mathbf{W} = n_1 \times n_o$
- $\mathbf{b}^{[1]} = n_1 \times 1$

- A **saída $\mathbf{h}$** da camada escondida pode então ser obtida por:
 - $\mathbf{h} = g(\mathbf{W}\mathbf{x} + \mathbf{b}^{[1]})$
 - Usamos g para representar a **função de ativação**
 - $\mathbf{h}$ será um vetor de dimensão $n_1 \times 1$, com mostra o exemplo a seguir

Matrizes da RNFF (6/13)

• Representação matricial

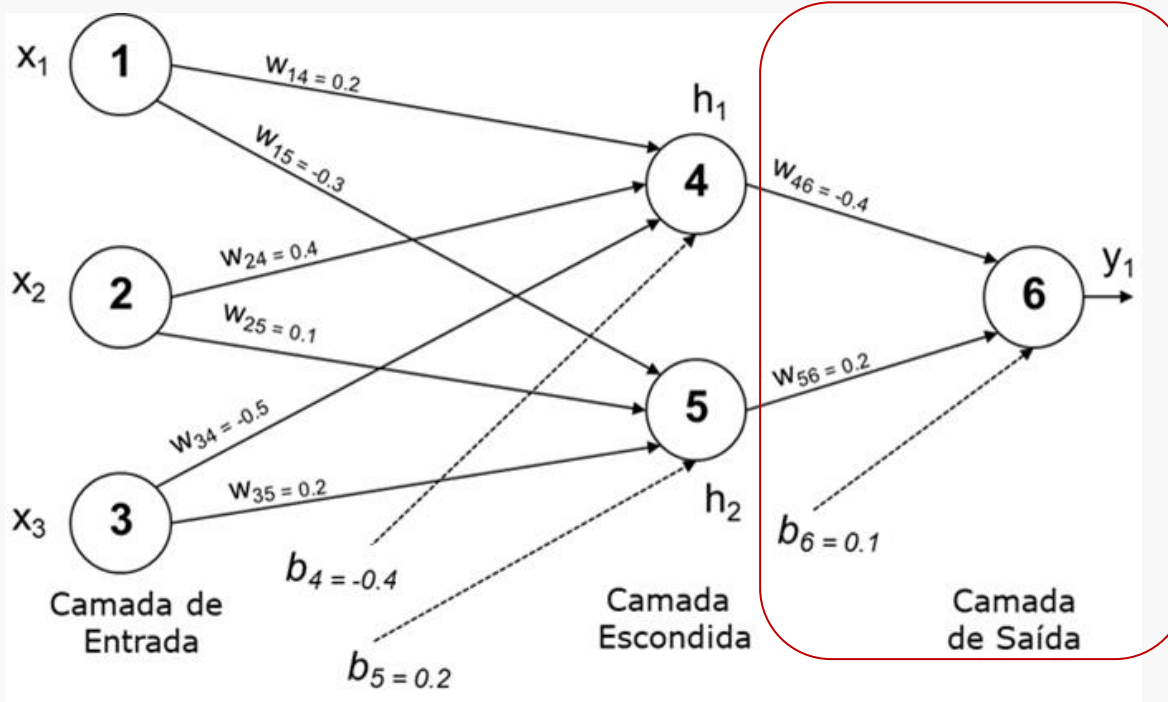
- **Exemplo** – Suponha que o vetor de entrada é $x = (1, 0, 1)$. Na prática x seria o novo objeto que queremos classificar com a RNFF treinada
- Suponha ainda que usaremos $g = \text{função sigmoide}$

- $h = g(Wx + b^{[1]})$

W			x	$b^{[1]}$
0.2	0.4	-0.5	1	-0.4
-0.3	0.1	0.2	0	0.2
			1	

- $h = g(\begin{vmatrix} 0.2 \times 1 + 0.4 \times 0 - 0.5 \times 1 \\ -0.3 \times 1 + 0.1 \times 0 + 0.2 \times 1 \end{vmatrix} + \begin{vmatrix} -0.4 \\ 0.2 \end{vmatrix})$
- $h = g(\begin{vmatrix} -0.7 \\ 0.1 \end{vmatrix})$
- $h = \begin{vmatrix} 0.332 \\ 0.525 \end{vmatrix}$
- **Obs.:** a multiplicação Wx é possível porque o número de colunas de W (3 colunas) e igual ao número de linhas de x (3 linhas)

Matrizes da RNFF (7/13)

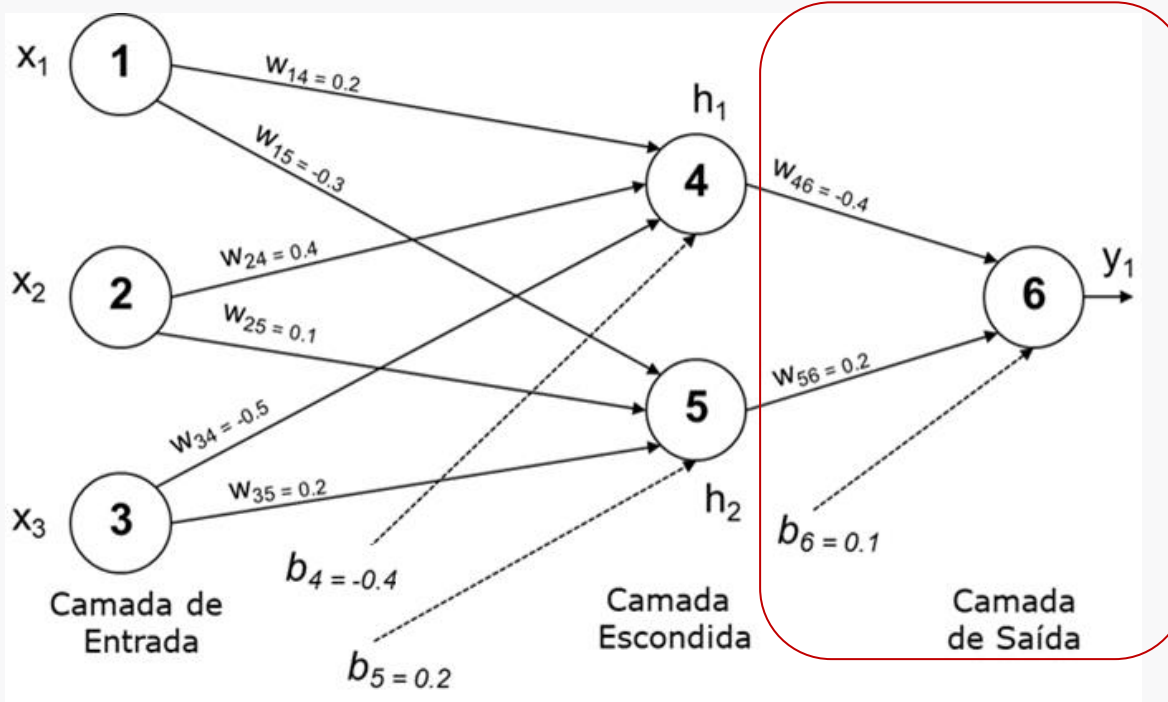


- $n_o = 3$
- $n_1 = 2$
- $n_2 = 1$
- $\mathbf{x} = n_o \times 1$
- $\mathbf{W} = n_1 \times n_o$
- $\mathbf{b}^{[1]} = n_1 \times 1$
- $\mathbf{h} = n_1 \times 1$
- $\mathbf{U} = n_2 \times n_1$
- $\mathbf{b}^{[2]} = n_2 \times 1$

• Parâmetros da camada saída

- $\mathbf{h}$: vetor saída da camada escondida (acabamos de ver) $\rightarrow$ dimensão $n_1 \times 1$
- $\mathbf{U}$: matriz de pesos. Cada elemento U_{ij} repres. o peso da conexão da unidade da camada escondida h_j p/ unidade de saída $y_i \rightarrow$ dimensão $n_2 \times n_1$
- $\mathbf{b}^{[2]}$: o vetor coluna de pesos da camada de saída $\rightarrow$ dimensão $n_2 \times 1$

Matrizes da RNFF (8/13)



- $n_o = 3$
- $n_1 = 2$
- $n_2 = 1$
- $\mathbf{x} = n_o \times 1$
- $\mathbf{W} = n_1 \times n_o$
- $\mathbf{b}^{[1]} = n_1 \times 1$
- $\mathbf{h} = n_1 \times 1$
- $\mathbf{U} = n_2 \times n_1$
- $\mathbf{b}^{[2]} = n_2 \times 1$

- A **saída y** da camada de saída pode então ser obtida por:
 - $\mathbf{y} = g(\mathbf{U}\mathbf{h} + \mathbf{b}^{[2]})$
 - A função de ativação **g** não precisa ser a mesma da camada escondida
 - **y** será um vetor $1 \times n_2$... nesse caso, 1×1 , pois a camada de saída tem apenas uma unidade (RN p/ classificação binária).

Matrizes da RNFF (9/13)

- Cálculo de y – saída da RN (gerada pela camada de saída)
 - Exemplo: continuação do exemplo anterior ... g = função sigmoide.
 - $y = g(Uh + b^{[2]})$

U		h	$b^{[2]}$
-0.4	0.2	0.332	0.1
		0.525	

- $y = g(\quad | -0.4 \quad \times 0.332 + 0.2 \times 0.525 \quad | + \quad | 0.1 | \quad)$
- $y = g(\quad | 0.0722 | \quad)$
- $y = \underline{0.5180}$ → essa é a saída da RNFF... *Mas como usar esse valor para fazer a classificação ???*

Matrizes da RNFF (10/13)

• Classificação com a Fronteira de Decisão

- Fizemos um exemplo de classificação binária (classe 0 ou classe 1)
- Porém, a saída gerou o valor **0.5180**. Porém, **como determinar a classe do novo objeto x** ?
- Método mais simples:
 - Dizemos que é classe=1, se a saída > 0.5
 - E dizemos classe=0 caso contrário
 - **0.5** é a **fronteira de decisão**:
 - decisão sobre $x = 1$, se saída > 0.5
 - decisão sobre $x = 0$, caso contrário
- Dessa forma, em nosso exemplo o novo objeto **x** seria classificado como **classe 1**
- *Nem sempre 0.5 é a melhor fronteira... Depende do problema...*

Matrizes da RNFF (11/13)

- **Resumo da Notação em Matrizes – Classificação Binária com uma camada escondida**
 - $\mathbf{h} = g_1(\mathbf{W}\mathbf{x} + \mathbf{b}^{[1]})$ → saída da camada escondida
 - $\mathbf{z} = \mathbf{U}\mathbf{h} + \mathbf{b}^{[2]}$ → saída intermediária da camada de saída
 - $\mathbf{y} = g_2(\mathbf{z})$ → saída final da RN
- g_1 e g_2 podem ser sigmoide, tanh, ReLU ou qualquer outra função de ativação.

Matrizes da RNFF (12/13)

- **IMPORTANTE – não há uniformidade na notação !**
 - Infelizmente, diferentes livros e ferramentas utilizam notações diferentes.
 - Por exemplo, alguns usam **g** para a função de ativação, outros usam **σ** , outros outra letra...
 - Entretanto, o que confunde mais é a divergência no formato das matrizes **W** e **U**.
 - Nesta aula, seguimos a notação do livro de Jurafsky e Martin (2025), onde:
 - Dimensão **W** = $n_1 \times n_o$
 - Dimensão **U** = $n_2 \times n_1$
 - Porém, há livros e ferramentas (como a sklearn) que adotam:
 - Dimensão **W** = $n_0 \times n_1$
 - Dimensão **U** = $n_1 \times n_2$

Matrizes da RNFF (13/13)

- **IMPORTANTE** – não há uniformidade na notação !

- Por exemplo, em nossa aula $W =$

0.2	0.4	-0.5
-0.3	0.1	0.2
- E fazemos a multiplicação Wx

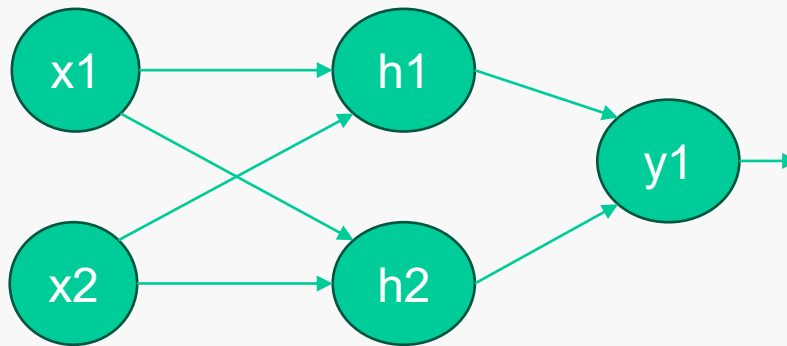
- Mas a sklearn gera $W =$

0.2	-0.3
0.4	0.1
-0.5	0.2

- E faz a multiplicação $W^t x$, onde W^t é a transposta da matriz W gerada pela sklearn

Exercício

- Para a RNFF já treinada abaixo, qual a classe do novo objeto $\mathbf{x} = (2, 3)$?
- Considere que:
 - A função de ativação nas camadas escondida e de saída* é a ReLU.
 - As matrizes W e U seguem o padrão descrito nos slides 32 a 40.



W		b^[1]	
0.5	0.0	-0.2	
-0.2	0.1	0.1	

U		b^[2]	
-0.4	0.2	0.3	

$$ReLU(z) = \max(z, 0)$$

* Na prática, a ReLU não costuma ser utilizada na camada de saída, mas apenas na(s) camada(s) escondida(s)

Referências

- JURAFSKY, D.; MARTIN, J. H. Neural Networks. *In*: JURAFSKY, D.; MARTIN, J. H. **Speech and Language Processing**. 3. ed. (Jan. 12, 2025 draft). [s.l.]: [s.n.], 2025. cap 7, p. 132 – 145. Disponível em: < <https://web.stanford.edu/~jurafsky/slp3/> >. Acesso em: 22 abr. 2025.
- HAN, J.; KAMBER, M.; PEI, J. Classification by Backpropagation. *In*: HAN, J.; KAMBER, M.; PEI, J. **Data Mining: Concepts and Techniques**. 3. ed. Waltham: Morgan Kaufmann, 2011. cap 9, p. 398-408.